SISTEMA DE GESTÃO DE FÉRIAS (TEMA 17)

Documentação Técnica, Validações SOLID, ACID & Trilha de Auditoria

> **Objetivo Acadêmico & Complexidade Real:** Evitar que o sistema seja um simples cadastro CRUD de datas e transformá-lo em um motor corporativo de regras de RH resiliente, auditável e seguro contra condições de corrida.

1. MÁQUINA DE ESTADOS FINITA E TRANSIÇÕES RESTRITAS

O `status` de uma solicitação de férias (*RASCUNHO, SOLICITADO, EM_ANALISE, APROVADO, RECUSADO, CANCELADO*) jamais é alterado de forma arbitrária via formulários ou JSON genérico. As modificações ocorrem obrigatoriamente acionando métodos de domínio específicos no backend.

A. Transições Orientadas a Ações (Domain-Driven)

Em vez de expor um endpoint estático como `PUT /solicitacoes/:id`, a API disponibiliza rotas explícitas ligadas às regras de negócio:

- `POST /ferias/:id/solicitar`
- `PATCH /ferias/:id/aprovar`
- `PATCH /ferias/:id/rejeitar`
- `PATCH /ferias/:id/cancelar`

B. Matriz de Permissões (RBAC por Estado)

- **Colaborador (Solicitante):** Pode acionar `solicitar()` em estado RASCUNHO e `cancelar()` em estado SOLICITADO. Proibido acionar aprovações ou rejeições.
- **Gestor do Setor / RH:** Pode executar `aprovar()`, `rejeitar()` ou solicitar ajustes quando a solicitação estiver em SOLICITADO ou EM_ANALISE. É expressamente vedada a auto-aprovação de férias.

C. Guardas de Transição (Transition Guards)

Validações síncronas que o backend obrigatoriamente executa antes de aceitar a transição de estado. Se qualquer validação falhar, uma exceção HTTP 400 (`BadRequestException`) é lançada e o estado permanece inalterado.

2. PRINCÍPIOS SOLID APLICADOS

A. Single Responsibility Principle (SRP)

O `Controller` é restrito a receber a requisição e emitir respostas HTTP. O `Service` orquestra a máquina de estados. As regras de validação (CLT e capacidade do setor) são totalmente isoladas em classes dedicadas de Guarda (*Guards*).

B. Open/Closed Principle (OCP)

O motor de transições está aberto para extensão e fechado para alteração. Novas regras de RH (ex: "proibição de férias no setor fiscal em dezembro") podem ser adicionadas criando novas classes de guarda sem alterar o código principal da máquina de estados.

C. Liskov Substitution Principle (LSP)

Todas as guardas concretas (`CltFractionGuard`, `TeamCapacityGuard`) implementam a interface comum `TransitionGuard`. O serviço executa qualquer guarda de forma polimórfica respeitando o mesmo contrato.

3. RASTREABILIDADE & TRILHA DE AUDITORIA (AUDIT LOG)

Nenhuma alteração de estado crítica pode simplesmente sobrescrever dados via `UPDATE` sem deixar rastro. Cada transição grava um registro imutável na tabela `historico_solicitacoes` contendo o estado anterior, novo estado, autor da ação, justificativa e carimbo de data/hora.

COLUNA	TIPO	DESCRIÇÃO / FUNÇÃO
`id`	TEXT (UUID)	Identificador único do registro de histórico
`solicitacao_id`	TEXT (FK)	Vínculo direto com a solicitação auditada
`estado_anterior`	Enum StatusFerias	Estado da solicitação antes da transição
`novo_estado`	Enum StatusFerias	Estado para o qual a solicitação transicionou
`autor_id`	TEXT (FK)	Identificador do funcionário/gestor executante
`justificativa`	TEXT	Motivo ou observação da aprovação/recusa/cancelamento
`timestamp`	TIMESTAMP	Data e hora exata da ocorrência

4. TRANSAÇÕES ACID & CONCORRÊNCIA (LOCK OTIMISTA)

Para evitar inconformidades e Race Conditions (ex: dois gestores aprovando requisições concorrentes e estourando o limite do setor), as operações são agrupadas em um bloco transacional `prisma.$transaction`:

- **Atomicidade (Tudo ou Nada):** A alteração do status, a atualização do saldo de dias e a gravação do histórico ocorrem juntas ou são totalmente revertidas via `ROLLBACK`.
- **Lock Otimista (`version`):** Cada atualização verifica e incrementa o campo `version` da solicitação, prevenindo sobreposições simultâneas.

5. MODELAGEM DO BANCO DE DADOS (SCHEMA.PRISMA)

Estrutura completa das entidades, relacionamentos e Enums dentro do schema dedicado `bi3041557`:

```
// prisma/schema.prisma
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  schemas  = ["bi3041557"]
}

enum StatusFuncionario {
  ATIVO
  INATIVO
}

enum Role {
  COLABORADOR
  GESTOR
  RH
  ADMIN
}

enum StatusFerias {
  RASCUNHO
  SOLICITADO
  EM_ANALISE
  APROVADO
  RECUSADO
  CANCELADO
  PAGO
}

model Funcionario {
  id           String            @id @default(uuid())
  nome         String
  email        String            @unique
  cargo        String
  role         Role              @default(COLABORADOR)
  setor        String            @default("Geral")
  dataAdmissao DateTime          @map("data_admissao")
  saldoDias    Int               @default(30) @map("saldo_dias")
  status       StatusFuncionario @default(ATIVO)
  gestorId     String?           @map("gestor_id")
  gestor       Funcionario?      @relation("GestorSubordinados", fields: [gestorId],
references: [id])
  subordinados Funcionario[]     @relation("GestorSubordinados")
  solicitacoes SolicitacaoFerias[]
  historicos   HistoricoSolicitacao[] @relation("AutorHistorico")
  createdAt    DateTime          @default(now()) @map("created_at")
  updatedAt    DateTime          @updatedAt @map("updated_at")

  @@map("funcionarios")
}

model SolicitacaoFerias {
  id              String         @id @default(uuid())
  funcionarioId   String         @map("funcionario_id")
  funcionario     Funcionario    @relation(fields: [funcionarioId], references: [id])
  dataInicio      DateTime       @map("data_inicio")
  diasSolicitados Int            @map("dias_solicitados")
  abonoPecuniario Boolean        @default(false) @map("abono_pecuniario")
  status          StatusFerias   @default(RASCUNHO)
  motivoRecusa    String?        @map("motivo_recusa")
  version         Int            @default(0)
  historicos      HistoricoSolicitacao[]
  createdAt       DateTime       @default(now()) @map("created_at")
  updatedAt       DateTime       @updatedAt @map("updated_at")

  @@map("solicitacoes_ferias")
```

6. IMPLEMENTAÇÃO DAS GUARDAS SOLID E SERVIÇOS

A. Interface da Guarda (`transition-guard.interface.ts`)

```
import { SolicitacaoFerias, Funcionario } from '@prisma/client';

export interface TransitionGuard {
  validate(
    solicitacao: SolicitacaoFerias,
    funcionario: Funcionario,
    novasDatas?: { dataInicio: Date; diasSolicitados: number },
  ): Promise<void>;
}
```

B. Guarda de Fracionamento CLT (`clt-fraction.guard.ts`)

```
import { BadRequestException, Injectable } from '@nestjs/common';
import { TransitionGuard } from './transition-guard.interface';
import { SolicitacaoFerias, Funcionario } from '@prisma/client';

@Injectable()
export class CltFractionGuard implements TransitionGuard {
  async validate(
    solicitacao: SolicitacaoFerias,
    funcionario: Funcionario,
    novasDatas?: { dataInicio: Date; diasSolicitados: number },
  ): Promise<void> {
    const dias = novasDatas ? novasDatas.diasSolicitados : solicitacao.diasSolicitados;

    if (dias < 5) {
      throw new BadRequestException(
        'Segundo a CLT, nenhum período de férias pode ser inferior a 5 dias corridos.',
      );
    }

    if (funcionario.saldoDias - dias < 0) {
      throw new BadRequestException(
        `Saldo insuficiente. O colaborador tem apenas ${funcionario.saldoDias} dias
disponíveis.`,
      );
    }
  }
}
```

C. Guarda do Limite de 20% do Setor (`team-capacity.guard.ts`)

```
import { BadRequestException, Injectable } from '@nestjs/common';
import { TransitionGuard } from './transition-guard.interface';
import { SolicitacaoFerias, Funcionario, StatusFerias } from '@prisma/client';
import { PrismaService } from '../../../database/prisma.service';

@Injectable()
export class TeamCapacityGuard implements TransitionGuard {
  constructor(private readonly prisma: PrismaService) {}

  async validate(solicitacao: SolicitacaoFerias, funcionario: Funcionario):
Promise<void> {
    const totalSetor = await this.prisma.funcionario.count({
      where: { setor: funcionario.setor, status: 'ATIVO' },
    });

    if (totalSetor === 0) return;

    const inicioSolicitado = new Date(solicitacao.dataInicio);
    const fimSolicitado = new Date(inicioSolicitado);
    fimSolicitado.setDate(fimSolicitado.getDate() + solicitacao.diasSolicitados);

    const ausentesAprovados = await this.prisma.solicitacaoFerias.count({
      where: {
        status: StatusFerias.APROVADO,
        funcionario: { setor: funcionario.setor },
        id: { not: solicitacao.id },
        AND: [{ dataInicio: { lte: fimSolicitado } }],
      },
    });

    const limitePermitido = Math.max(1, Math.floor(totalSetor * 0.2));

    if (ausentesAprovados >= limitePermitido) {
      throw new BadRequestException(
        `Limite do setor atingido: O setor '${funcionario.setor}' já tem o limite de
${limitePermitido} colaborador(es) (20%) ausente(s) no período selecionado.`,
      );
    }
  }
}
```

7. REGISTRO DOS TESTES EFETUADOS

Todas as travas e regras de negócio foram devidamente testadas e validadas no ambiente de execução:

1. Teste de Validação CLT

Ao submeter um pedido de férias com duração de 2 dias, o sistema bloqueou a requisição e retornou com sucesso a mensagem: *"Segundo a CLT, nenhum período de férias pode ser inferior a 5 dias corridos."*

2. Teste de Transação ACID & Audit Log

Uma solicitação de 17 dias foi aprovada. Em uma única transação, o status mudou para `APROVADO`, o saldo do colaborador no banco caiu de 30 para 13 dias e a transição `SOLICITADO → APROVADO` foi registrada no Audit Log. Posteriormente, ao cancelar, os 17 dias foram estornados automaticamente ao saldo.

3. Teste da Regra dos 20% da Equipe

Com 5 colaboradores cadastrados no mesmo setor ('Geral'), o limite de 20% autorizou apenas 1 ausência simultânea. A tentativa de aprovar uma segunda solicitação no mesmo período disparou a exceção: *"Limite do setor atingido: O setor 'Geral' já tem o limite de 1 colaborador(es) (20%) ausente(s)..."*

8. INSTRUÇÕES PARA ENTREGA E AVALIAÇÃO

A. Estrutura do Arquivo .gitignore

Garante que pastas pesadas e dados sensíveis fiquem fora do repositório/zip:

```
node_modules/
.next/
dist/
build/
.env
```

B. Arquivo de Exemplo de Variáveis (.env.example)

```
DATABASE_URL="postgresql://usuario:senha@10.102.55.11:5432/postgres?schema=bi3041557"
PORT=8000
```

C. Script do AI-LOG.md (Exigência Acadêmica)

O arquivo `AI-LOG.md` deve constar na raiz do repositório descrevendo o processo de apoio da IA, as falhas identificadas em abordagens iniciais (como `db push --force-reset` sem permissões de superuser no banco compartilhado) e as soluções aplicadas através de SQL direto e guardas SOLID.